

DATAGIT

Full Architecture, AI Runtime, Integration & Deployment Guide

LangChain + LangGraph + LangSmith edition

Simple-language production planning document for the DATAGIT team

Purpose: This document turns the full DATAGIT idea into one practical architecture: local desktop app + special frontend + special backend + LangChain/LangGraph AI runtime + LangSmith for development/observability + Git/DVC/VS Code integration + production packaging and distribution.

Important: This guide keeps all 19 requested topics and the team responsibilities exactly as specified. Where a technology choice is still a design decision, it is clearly marked as a recommendation rather than a finished implementation fact.

Executive Summary

DATAGIT is a local-first ML development tracking, comparison, investigation, and reporting product. The user works with their own local Git, DVC, VS Code, code, datasets, and models. DATAGIT adds a unified application layer around that environment.

USER MACHINE

DATAGIT Desktop App

|

v

DATAGIT Backend / Core

|\\

| _ Git + DVC + Project State + Evidence + Database

|

v

AI Runtime

|

+-- LangGraph (agent orchestration / state / execution)

|

+-- LangChain (model + tool + structured-output building blocks)

|

+-- 6 DATAGIT agents

|

Data -> Code -> Experiment -> Model

|

-> Investigation -> Report

|

+-- LangSmith (development/evaluation/observability, optional in customer runtime)

|

v

DATAGIT Backend

|

v

Frontend display / reports / copilot

Core design rule: DATAGIT Backend/Core owns the authoritative project state, Git/DVC facts, official comparisons, evidence, and report records. AI interprets validated evidence; AI does not become the source of truth.

Contents

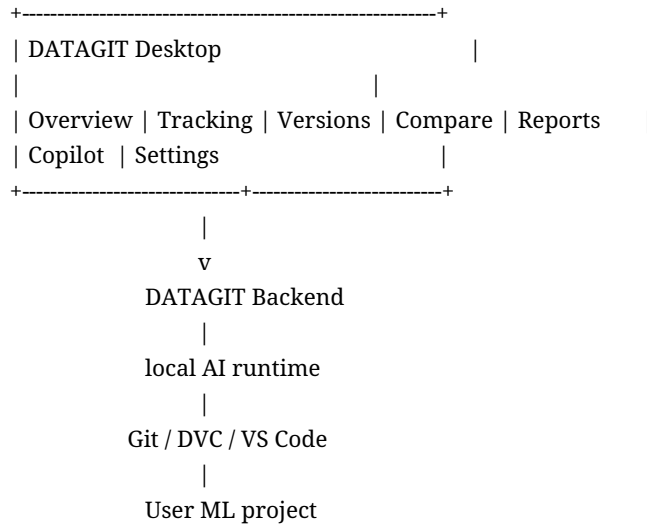
1. DATAGIT CLI desktop app on local device
2. DATAGIT CLI desktop app on local device - special frontend
3. DATAGIT CLI desktop app on local device - special backend
4. DATAGIT CLI desktop app on local device - special LangChain layer
5. Internal workflow at the app
6. Internal workflow at LangChain, LangGraph, LangSmith
7. How app sends input to LangChain, LangGraph, LangSmith

8. 8. How LangChain, LangGraph, LangSmith send results to app
9. 9. Connection between app and LangChain, LangGraph, LangSmith
10. 10. How LangChain, LangGraph, LangSmith work with Git, DVC, VS Code, and DATAGIT
11. 11. All about AI agents - the AI Agent edition
12. 12. How this works on the user local device
13. 13. How we deploy this at production level
14. 14. What we provide the user to install and setup
15. 15. How we deploy so anyone can access and use it locally
16. 16. Extra tips + team responsibilities + recommended architecture/stack
17. 17. User flow
18. 18. Internal flow
19. 19. Input / Output contracts
20. 20. Production readiness checklist
21. 21. Current verified technology notes and sources

1. DATAGIT CLI Desktop App on Local Device

The final user product should feel like a normal application, not like a collection of developer tools. The user downloads one DATAGIT installer. DATAGIT then runs its own desktop interface and local services.

USER COMPUTER



- DATAGIT runs locally and works with the user's local project rather than requiring the project to live on your company server.
- Git remains responsible for code history. DVC remains responsible for dataset/artifact version history. VS Code remains the development environment.
- DATAGIT adds project state, cross-domain evidence, comparisons, investigations, reports, and a single product experience.
- The desktop app should start the required local services, check health, and hide implementation details from the normal user.

Recommended product boundary: Do not make DATAGIT look like an n8n/LangGraph administration console. The user should see DATAGIT concepts such as Project, Version, Compare, Report, Tracking, and Copilot.

2. DATAGIT CLI Desktop App on Local Device - Special Frontend

The special frontend is the user-facing part of DATAGIT. It is not responsible for Git/DVC logic or AI reasoning. It asks the backend for data and displays validated results.

Frontend area	What it does	What it should not do
---------------	--------------	-----------------------

Overview	Shows project health, latest version, current state, recent activity.	Should not calculate authoritative Git/DVC facts.
Tracking	Shows meaningful changes/events and project history.	Should not treat every file save as a new DATAGIT Version.
Versions	Shows DATAGIT Versions made from connected project state.	Should not create version identity by AI.
Compare	Lets user choose two versions and ask questions.	Should not compute official deltas itself.
Reports	Displays validated reports, evidence, confidence, and limitations.	Should not rewrite evidence into new facts.
Copilot	Collects user questions and starts investigation/report tasks.	Should not call the AI provider directly if backend is the security boundary.
Settings	AI provider, project path, local services, updates, logs, permissions.	Should not expose internal runtime administration unnecessarily.

3. DATAGIT CLI Desktop App on Local Device - Special Backend

The special backend is the most important deterministic part of the product. It is the bridge between the UI, local Git/DVC/VS Code environment, DATAGIT storage, and the AI runtime.

FRONTEND

```
|
| authenticated local request
v
```

DATAGIT BACKEND

```
|
+-- Project State
+-- Git integration
+-- DVC integration
+-- Evidence collection
+-- Official comparisons / deltas
+-- DATAGIT Versions
+-- Database
+-- Report storage
+-- AI request adapter
+-- validation / provenance
|
```

v

LANGGRAPH AI RUNTIME

- Accept and validate frontend input.
- Identify the project and project state.
- Read Git and DVC evidence from the local project using controlled operations.
- Create the DATAGIT Version / Project State bundle.
- Calculate official deterministic differences before AI runs.
- Send only the validated evidence/context needed by the AI runtime.
- Validate AI output before storing/displaying it.
- Persist reports, findings, execution status, and provenance.
- Return clean structured data to the frontend.

4. DATAGIT CLI Desktop App on Local Device - Special LangChain Layer

LangChain is not the DATAGIT product. It is a software library layer used inside DATAGIT's AI implementation. The user should not manually install LangChain. Our product packages the required dependencies.

DATAGIT AI CODE

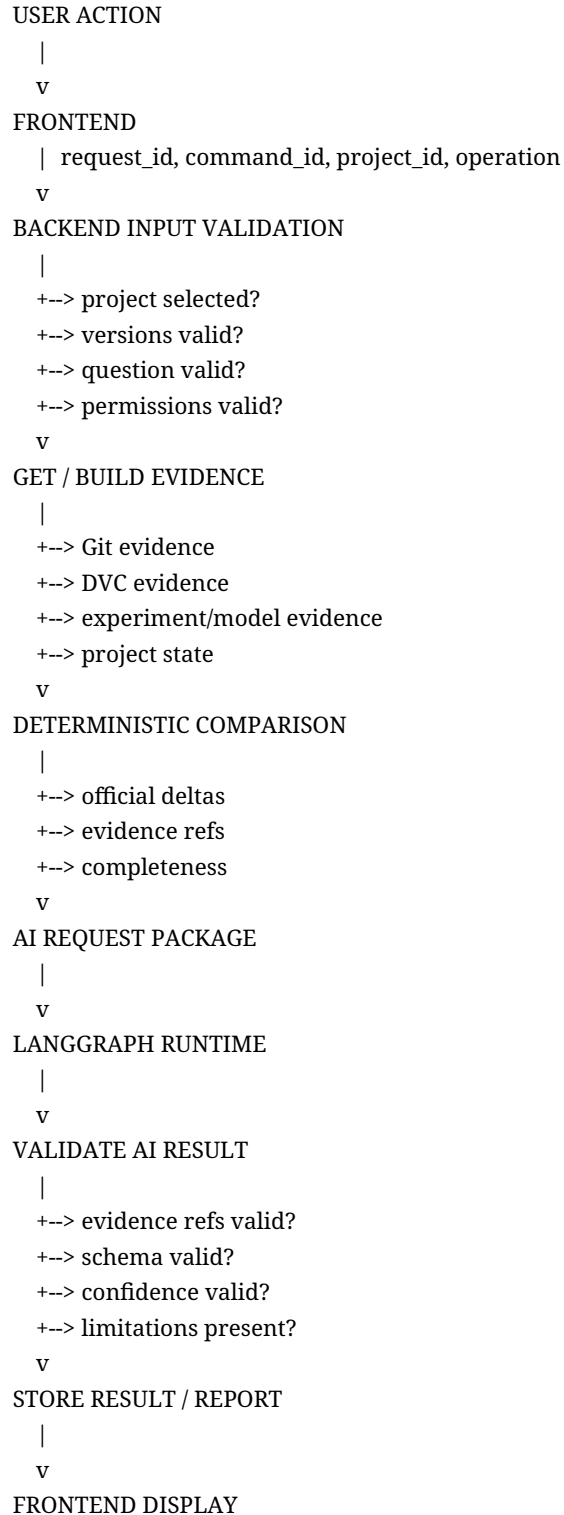
```

|
+-- LangChain model integrations
+-- LangChain structured output
+-- LangChain tool interfaces
+-- provider adapters
|
v
LLM provider or local model
|
v
structured agent result

```

- LangChain provides reusable model/tool/structured-output building blocks.
- Our prompts, business rules, schemas, evidence rules, validation rules, and agent responsibilities remain DATAGIT code.
- LangGraph should control multi-agent execution and state; LangChain is used inside the agent implementations where it simplifies model/tool interactions.
- LangSmith should not be required for the normal user to understand or manually operate.

5. Internal Workflow at the App



Key point: The backend prepares the truth first; the AI receives structured evidence second. This prevents the AI layer from silently becoming the project database.

6. Internal Workflow at LangChain, LangGraph, LangSmith

Recommended division: LangGraph is the main AI runtime/orchestrator; LangChain is used inside agents for model and tool building blocks; LangSmith is the observability/evaluation layer for development and, if enabled, production monitoring.

LANGGRAPH GRAPH

START

|

v

Prepare state

|

+-----+-----+-----+-----+

| | | |

Data Code Experiment Model (parallel where useful)

| | | |

+-----+-----+-----+-----+

|

v

Validate specialist results

|

v

Investigation Agent

|

findings / hypotheses

|

v

Report Agent

|

v

END

At the agent level:

LangGraph node -> LangChain -> model/tool -> structured result

LangSmith (optional tracing/evaluation):

run -> trace -> evaluation / debugging -> engineering feedback

- LangGraph stores and moves the state required by the graph and supports controlled branching, retries, checkpoints, and long-running execution patterns.
- LangChain is used when an agent needs model invocation, structured output, tools, provider integrations, or similar building blocks.
- LangSmith can trace runs, support evaluation datasets, and help the team debug agent behavior. It is not the source of truth for DATAKIT state.
- Customer deployments should not depend on a full self-hosted LangSmith platform unless the commercial/operational plan explicitly requires it.

7. How App Sends Input to LangChain, LangGraph, LangSmith

The app should not send random UI text directly into an agent. The backend creates a controlled request envelope first.

```
{
  "request_id": "REQ-...",
  "command_id": "CMD-...",
  "project_id": "PROJECT-...",
  "operation": "investigate_model",
  "question": "Why did model-v2 perform better than model-v1?",
  "project_state_ref": "STATE-...",
  "scope": {
    "model_a": "model-v1",
    "model_b": "model-v2"
  },
  "evidence": { ...validated DATAGIT evidence... }
}
```

- Frontend sends input to DATAGIT Backend.
- Backend authenticates and validates it.
- Backend resolves the project/version scope.
- Backend collects deterministic evidence.
- Backend creates the AI state/request.
- Backend invokes the LangGraph runtime.
- LangGraph routes data to the appropriate agent nodes.
- LangChain handles the agent's model/tool calls.
- LangSmith, when enabled, receives traces asynchronously according to the configured privacy and observability policy.

8. How LangChain, LangGraph, LangSmith Send to App

```
LLM / tools
|
v
LangChain
| structured agent output
v
LangGraph state
| final graph result
v
DATAGIT Backend
|
+--> schema validation
+--> evidence-ref validation
+--> confidence / limitation checks
+--> persistence
v
```

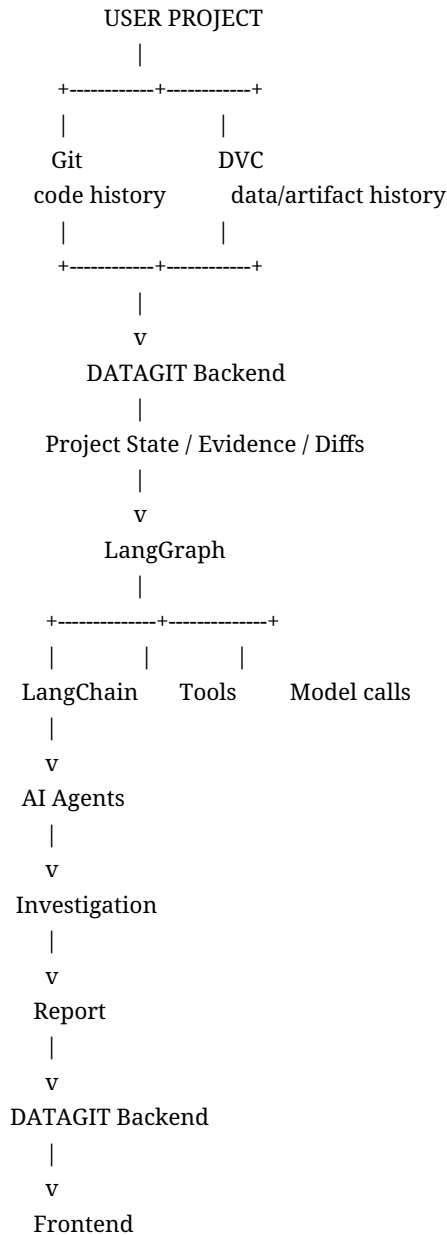
Important: LangSmith should not be the primary response channel to the user interface. It is for tracing/evaluation/monitoring. The normal product response path is LangGraph -> Backend -> Frontend.

9. Connection Between App and LangChain, LangGraph, LangSmith

Connection	Recommended form	Reason
Frontend -> Backend	Local authenticated HTTP/API	Keeps UI separate from internal services.
Backend -> LangGraph	Local authenticated service API / Agent Server API	Lets backend control requests and validate context.
Agent -> LLM	LangChain provider adapter	Keeps model integration inside the AI layer.
LangGraph -> LangSmith	SDK/callback tracing when enabled	Observability without making LangSmith the product data store.
Backend -> Git/DVC	Controlled local process/tool interface	Preserves Git/DVC as source-of-truth systems.
DATAGIT -> VS Code	Local application launch / optional extension bridge	VS Code remains the development environment.

Security boundary: The frontend should not talk directly to raw Git/DVC/LLM/provider credentials. The backend is the local security and policy boundary.

10. How LangChain, LangGraph, LangSmith Work Along With Git, DVC, VS Code, and DATAGIT



VS Code sits beside this flow and is the user's local development environment.

- Git: authoritative code history.
- DVC: authoritative dataset/artifact version history.
- VS Code: development environment and optional event/activity signal source.
- DATAGIT Backend: unified project state, evidence, comparisons, provenance, reporting, storage, and validation.
- LangGraph: agent orchestration/runtime.
- LangChain: agent model/tool building blocks.
- LangSmith: optional observability/evaluation layer.

11. All About AI Agents - AI Agent Edition

Each agent is a controlled software component with a clear input schema, evidence contract, job, output schema, validation rules, and provenance rules.

Agent	Input	Job	Output
Data Agent	Dataset/DVC evidence	Schema/profile/quality changes	Data findings + confidence + limitations
Code Agent	Git commits/diffs/config evidence	Relevant code/config changes	Code findings + hypotheses + refs
Experiment Agent	Experiment runs/metrics/parameters	Run outcomes and parameter changes	Experiment findings + tradeoffs
Model Agent	Model comparison evidence	Model metric/resource tradeoffs	Model findings + limitations
Investigation Agent	Validated specialist results + project state	Cross-domain correlation and root-cause hypotheses	Findings + hypotheses + confidence + open questions
Report Agent	Approved investigation + evidence refs	Readable project report	Structured report / narrative

11.1 Common agent input contract

```
{
  "request_id": "REQ-...",
  "project_id": "PROJECT-...",
  "project_state_ref": "STATE-...",
  "question": "...",
  "scope": { ... },
  "evidence": { ... },
  "evidence_refs": ["EV-...", "CMP-..."],
  "limitations": [ ... ]
}
```

11.2 Common agent output contract

```
{
  "findings": {
    "facts": [],
    "observations": [],
    "interpretations": [],
    "hypotheses": []
  }
}
```

```

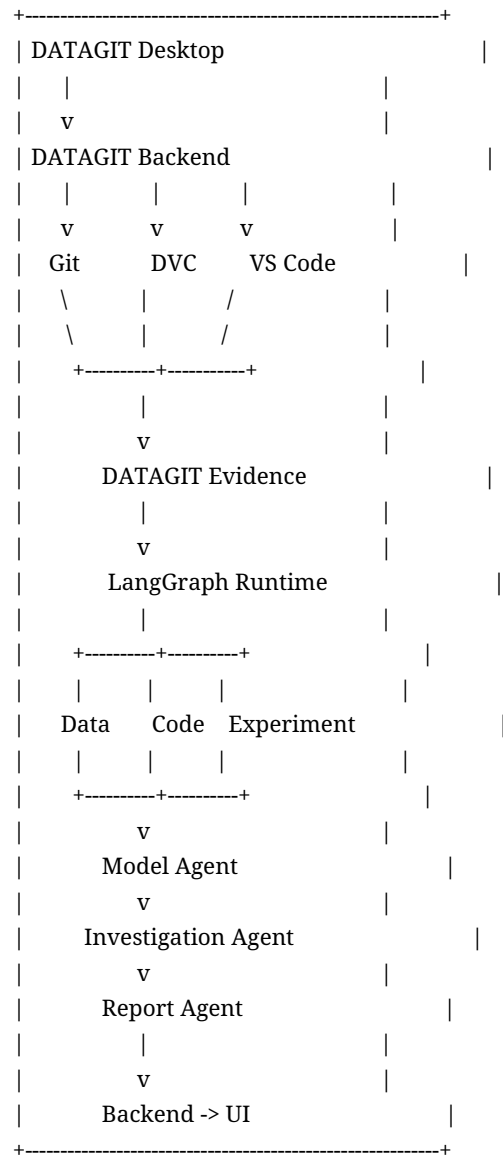
},
"confidence": "HIGH|MEDIUM|LOW|LIMITED",
"confidence_rationale": "...",
"evidence_refs": [],
"insufficient_evidence": [],
"limitations": [],
"provenance": []
}

```

- Facts must be directly supported by supplied evidence.
- Official numbers and diffs must come from DATAGIT deterministic computation.
- AI must not silently invent provenance or evidence IDs.
- If evidence conflicts, surface the conflict.
- If evaluation split information is unavailable, say so.
- If datasets differ, treat that as a major limitation and cap confidence.
- Tradeoffs should remain tradeoffs, not simplified verdicts.
- AI output is invalid until the backend/agent validation layer accepts it.

12. How This Works on the User Local Device

USER PC



- The user installs DATAGIT, not LangChain manually.
- The user's Git, DVC, VS Code, and ML projects remain local.
- DATAGIT creates/records unified Project State from Git/DVC/experiment/model evidence.
- The AI runtime receives validated context and returns structured results.
- If a cloud LLM is selected, the relevant prompt/evidence leaves the machine to that provider; local models can be added later.
- If AI runtime is temporarily unavailable, core DATAGIT tracking should remain usable in a degraded mode.

13. How We Deploy This at Production Level

Production deployment has two sides: our company infrastructure and the customer's local installation.

Company side	User side
Website / product pages	DATAGIT desktop application
Download/release server	DATAGIT Backend
CI/CD build pipeline	LangGraph/LangChain AI runtime
Code signing and release	Local DATAGIT database/state
Documentation/support	User Git/DVC/VS Code/project
Optional LangSmith observability	Optional LangSmith tracing, if policy allows

```

COMPANY
GitHub / CI-CD
|
v
Build DATAGIT
|
+-- Desktop
+-- Backend
+-- AI runtime
+-- installer
|
v
Signed Release
|
v
DATAGIT.COM / Release CDN
|
v
User downloads installer
|
v
User PC -> full local DATAGIT stack

```

Production rule: Do not make the customer install Python packages, clone the repo, or manually configure the internal agent graph. The installer must turn the developer build into a repeatable user setup.

14. What We Provide the User to Install and Setup

The normal user experience should be one download and one setup wizard.

```

WEBSITE
|
v
[ Download DATAGIT for Windows ]
|
v
DATAGIT-Setup.exe
|
+--> DATAGIT Desktop
+--> DATAGIT Backend
+--> LangChain / LangGraph runtime dependencies
+--> DATAGIT AI agents
+--> local configuration / persistence
+--> launcher / updater
|
v
SETUP WIZARD
|
+--> detect Git
+--> detect DVC
+--> detect VS Code
+--> choose project folder
+--> configure AI provider
+--> test local services
+--> health check
|
v
DATAGIT READY

```

- The user should not manually download LangChain or LangGraph.
- The user should not manually configure internal agent code.
- DATAGIT should detect existing Git/DVC/VS Code rather than blindly replacing them.
- Secrets must be stored securely and never shipped inside public workflow/source packages.
- The setup wizard should clearly report missing prerequisites and provide safe next actions.
- For Windows-first deployment, use a signed installer and an updater; plan macOS/Linux packaging after Windows is stable.

14.1 User setup responsibilities

- Accept installation/license terms.
- Choose or confirm the default DATAGIT data location.
- Connect one or more local ML project folders.
- Confirm Git/DVC/VS Code are available.
- Choose the AI provider and enter the provider credential if required.
- Choose privacy/telemetry options.
- Run the setup health check and finish.

15. How We Deploy So Anyone Can Access and Use It Locally

PUBLIC DISTRIBUTION

DATAGIT.COM

```
|
+-- Features
+-- Documentation
+-- Download
+-- Support
```

```
|
v
```

DATAGIT Release

```
|
+-- signed installer
+-- checksum / integrity metadata
+-- version notes
```

```
|
v
```

USER

```
|
+-- download
+-- install
+-- connect local project
+-- use DATAGIT
```

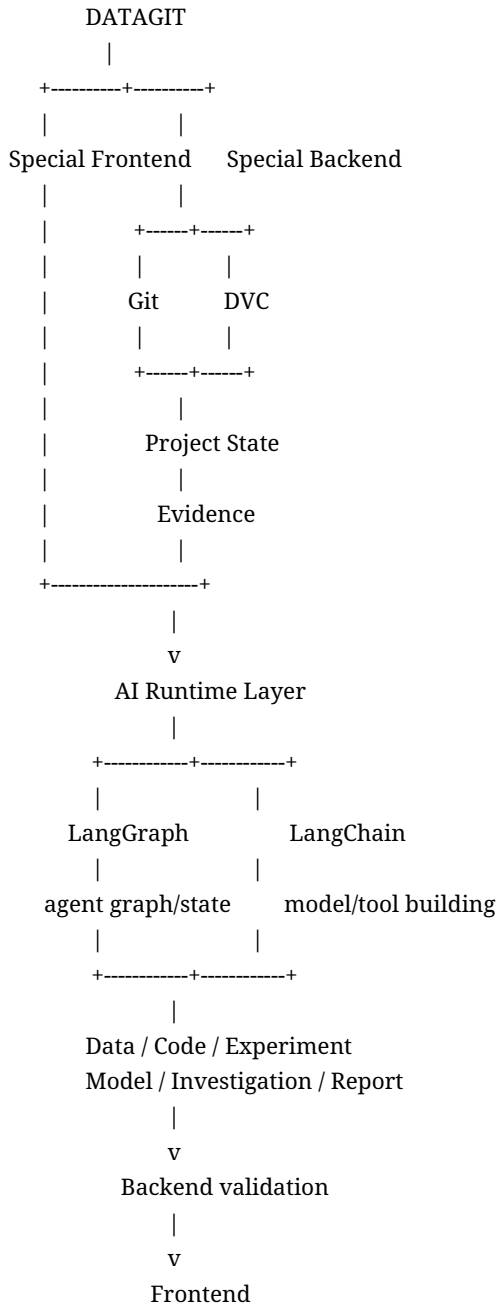
UPDATES

User DATAGIT -> checks supported release -> downloads update ->
 backup user state -> migrate if needed -> health check -> restart

- Use a stable download URL plus a release archive/CDN.
- Publish versioned installers and release notes.
- Sign the installer so Windows users can verify the publisher.
- Have CI/CD produce the same artifact that was tested.
- Keep user project data separate from application binaries.
- Provide uninstall/repair/update paths that preserve user data by default.
- Provide offline/degraded behavior for core local tracking where practical; cloud AI still requires network access unless a local model is used.

16. Extra Tips + Team + Recommended Architecture, Integration, Tools, and Stack

16.1 Recommended internal architecture



LangSmith = observability/evaluation
(not the DATAGIT source of truth)

16.2 Suggested internal components to build

Component	What we build
Project service	Register projects, paths, configuration, health.

Git adapter	Read controlled Git history/diff/state.
DVC adapter	Read controlled DVC state/diff/metadata.
Project State service	Define and persist immutable meaningful project states.
Evidence service	Create deterministic evidence bundles and refs.
Comparison service	Compute official deltas and compatibility flags.
AI adapter	Convert backend request into LangGraph run input.
AI validation service	Validate structured agent outputs and evidence refs.
Report service	Persist reports, status, provenance, and limitations.
Local service manager	Start/stop/health-check local runtime components.
Update manager	Detect/download/apply supported DATAGIT updates.
Installer	Package the tested product into a normal-user setup.

16.3 Recommended technical stack

Area	Recommendation	Why
Desktop shell	A lightweight desktop shell around the special frontend (for example Tauri or Electron)	Gives a normal desktop-app experience while keeping local access.
Frontend	React + TypeScript	Good fit for the existing UI direction and typed frontend state.
Backend	Python + FastAPI	Matches the ML ecosystem and local tool integration well.

Database	SQLite for simple local app data, or PostgreSQL if the chosen Agent Server architecture truly requires it	Start simple; add heavier infrastructure only when needed.
Agent runtime	LangGraph	Stateful agent orchestration/runtime.
AI building blocks	LangChain	Model/tool/structured-output components.
Observability	LangSmith	Agent tracing/evaluation/debugging; optional customer runtime.
Version/data tools	Git + DVC	Keep the user's authoritative history in the tools they already use.
Packaging	Signed Windows installer + updater	Normal-user distribution.
CI/CD	GitHub Actions or equivalent	Repeatable build/test/release process.
Testing	pytest + agent evaluation datasets + integration tests	Deterministic + AI quality testing.
Security	OS secret store / secure credential manager + least privilege	Protect API keys and local access.

Important stack note: LangGraph can be used without LangChain, but DATAGIT should use LangChain where its model/tool abstractions materially reduce code. Avoid adding framework layers that do not solve a real problem.

16.4 Team responsibilities

Person	Responsibility
Samarth	DATAGIT frontend / CLI interface. Manage all output display and handle the user interface.
Varad	DATAGIT backend. Manage backend work for getting all user input, Project State, Git, DVC, Evidence, Versions, Database; pass required

	context to the AI layer; receive AI output; validate/format it; send it to frontend; support required integration pieces. The team can also retain n8n-related compatibility work if it remains necessary.
Soham	Main engine. Handle LangChain, LangGraph, LangSmith integration and the AI Layer: Data Agent, Code Agent, Experiment Agent, Model Agent, Investigation Agent, Report Agent.
Swati	Deployment, mentoring, release management, and the bridge between the user and the team: how users download, install, setup, update, repair, and use DATAGIT locally.

16.5 External connections

- User local Git and DVC repositories.
- User local VS Code installation/workspace.
- AI provider APIs or local inference runtime.
- Optional LangSmith cloud/self-hosted observability depending on privacy and commercial plan.
- Company website, download server/CDN, support system, and release metadata.
- Optional cloud account/subscription service if DATAGIT later adds teams, billing, synchronization, or hosted features.

16.6 Extra deployment tips

- Never make the frontend directly control secrets or arbitrary local commands.
- Never make AI the authoritative versioning system.
- Keep application binaries separate from user project data and local state.
- Use request_id and command_id across the entire local request lifecycle.
- Use idempotency so a repeated request does not accidentally create duplicate versions or reports.
- Have explicit degraded states: core DATAGIT can remain usable when AI is unavailable.
- Log enough to debug failures without logging sensitive credentials or raw datasets by accident.
- Pin compatible dependency versions and maintain a tested software bill of materials.
- Before commercial release, review licensing/terms for every third-party component and any optional managed service.
- Do not ship a full LangSmith self-hosted platform inside every user installation. Treat that as an enterprise/cloud infrastructure decision.

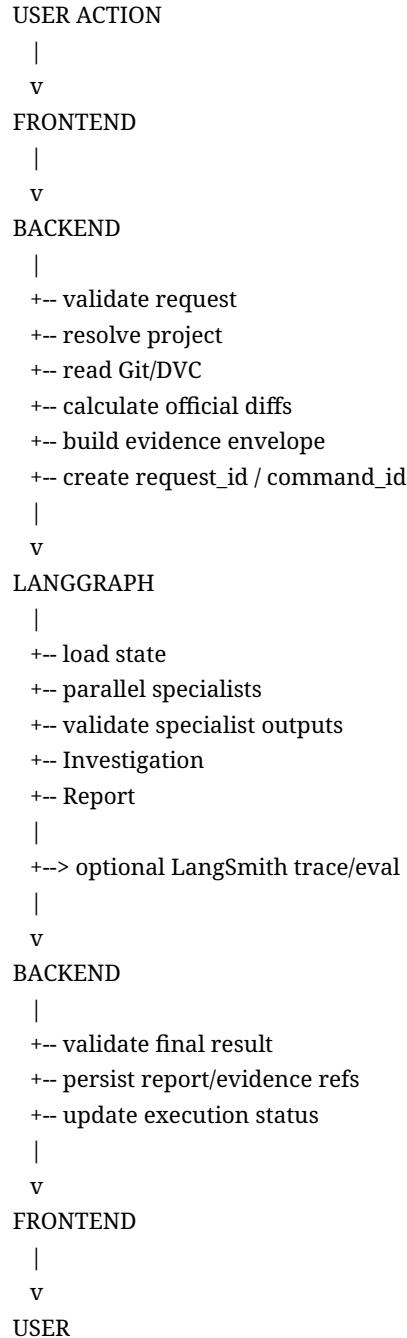
17. User Flow

1. User visits DATAGIT website
2. User downloads DATAGIT installer
3. User installs DATAGIT
4. Setup checks Git / DVC / VS Code
5. User opens DATAGIT
6. User adds/selects local ML project
7. DATAGIT registers project
8. User works/trains model in VS Code
9. User creates a meaningful DATAGIT Version
10. Backend reads Git + DVC + project state
11. Backend creates validated evidence
12. User asks a comparison/investigation question
13. LangGraph runs specialist agents
14. Investigation Agent correlates evidence
15. Report Agent creates the report
16. Backend validates/stores result
17. Frontend displays version / compare / report
18. User continues work in VS Code
19. Next meaningful state creates next DATAGIT Version

User responsibilities should stay simple

- Install DATAGIT.
- Connect their project.
- Keep Git/DVC/VS Code configured correctly.
- Provide AI provider credentials if required.
- Use DATAGIT to create versions and ask questions.
- Review reports and limitations before making engineering decisions.

18. Internal Flow



No silent AI authority: If evidence is missing, agents should say it is missing. If evidence conflicts, the report should say it conflicts. If AI output fails validation, the backend marks the AI step failed rather than accepting malformed output.

19. Input / Output

19.1 Frontend -> Backend input

```
{
  "request_id": "REQ-...",
  "command_id": "CMD-...",
  "project_id": "PROJECT-...",
  "operation": "compare_versions",
  "version_a": "VERSION-001",
  "version_b": "VERSION-002",
  "question": "Why did accuracy improve?"
}
```

19.2 Backend -> AI runtime input

```
{
  "request_id": "REQ-...",
  "command_id": "CMD-...",
  "project_id": "PROJECT-...",
  "project_state_ref": "STATE-002",
  "question": "Why did accuracy improve?",
  "scope": {
    "version_a": "VERSION-001",
    "version_b": "VERSION-002"
  },
  "evidence": {
    "data": { ... },
    "code": { ... },
    "experiment": { ... },
    "model": { ... }
  },
  "evidence_refs": ["CMP-...", "DATA-...", "EXP-...", "MODEL-..."]
}
```

19.3 AI runtime -> Backend output

```
{
  "request_id": "REQ-...",
  "command_id": "CMD-...",
  "project_id": "PROJECT-...",
  "status": "success | degraded | failed | insufficient_evidence",
  "findings": {
    "facts": [],
    "observations": [],
    "interpretations": [],
    "hypotheses": []
  },
  "confidence": "HIGH | MEDIUM | LOW | LIMITED",
  "evidence_refs": [],
  "limitations": [],
  "provenance": [],
}
```

```
"report_ref": "REPORT-..."  
}
```

19.4 Backend -> Frontend output

```
{  
  "project": { ... },  
  "version_a": { ... },  
  "version_b": { ... },  
  "comparison": { ... },  
  "findings": [ ... ],  
  "confidence": "MEDIUM",  
  "limitations": [ ... ],  
  "report_ref": "REPORT-...",  
  "status": "success"  
}
```

Contract rule: Do not invent fields simply because a downstream agent would like them. Backend contracts should be designed and versioned deliberately. All AI-specific fields should have schema tests.

20. Production Readiness Checklist

Product

- One installer works for a clean supported machine.
- One normal user can add a local project without cloning our source repo.
- Frontend does not require n8n/LangGraph administration knowledge.
- DATAGIT remains usable when AI is temporarily unavailable.

Backend

- Project State is defined and immutable.
- Git/DVC adapters are tested on clean and real repositories.
- Official diffs are deterministic.
- AI input/output contracts are versioned and validated.
- Authentication/authorization exists for local service calls.
- Idempotency prevents duplicate version creation.

AI

- All six agents have deterministic input fixtures.
- All six agents have output schema validation.
- All causal language is constrained by evidence.
- Evidence references are validated.
- Missing evaluation split is reported.
- Dataset mismatch caps confidence.
- Conflicting evidence is surfaced.
- Investigation is not duplicated in Report Agent.

LangGraph

- Graph state is explicitly defined.
- Retries and failure states are defined.
- Checkpoint/persistence strategy is defined.
- Long-running runs can recover safely.
- Agent tools are allowlisted.

LangSmith

- Tracing policy is explicit.
- Sensitive-data handling is documented.
- Tracing can be disabled.
- Evaluation datasets are versioned.
- Customer deployment does not silently depend on hosted tracing.

Deployment

- Installer is signed.
- Application data is separate from binaries.

- Update/rollback strategy is tested.
- Uninstall preserves user data by default.
- Logs are collected safely.
- Health checks exist.
- Clean-machine install test is automated.
- Windows firewall/network behavior is tested.
- Third-party licensing has been reviewed.

21. Current Verified Technology Notes and Sources

The following notes were checked against current official sources during preparation of this document. These are technology facts that can change over time, so the team should re-check them before the commercial release.

- LangGraph is a low-level orchestration framework for building, managing, and deploying long-running, stateful agents; its GitHub repository currently identifies the project as MIT licensed.
- LangGraph can be used without LangChain, but DATAGIT can use LangChain where its abstractions are useful.
- Current LangSmith pricing includes Developer, Plus, and Enterprise plans; current pricing materials list self-hosted/hybrid deployment options under Enterprise.
- Current LangSmith documentation describes evaluation/observability for LangGraph and other AI frameworks and supports cloud or enterprise self-hosting options.
- Current LangSmith self-hosted licensing is an Enterprise offering, so the team should not assume a full customer-local LangSmith platform is a free component of DATAGIT.

Official references

Source	URL
LangGraph GitHub / license	https://github.com/langchain-ai/langgraph
LangSmith plans and pricing	https://www.langchain.com/pricing
LangSmith evaluation / observability	https://www.langchain.com/langsmith/evaluation
LangSmith self-hosted license guidance	https://support.langchain.com/articles/7011309930-how-do-i-obtain-a-self-hosted-langsmith-license-key
LangChain AI organization / source repositories	https://github.com/langchain-ai

Final Recommended DATAGIT Architecture

